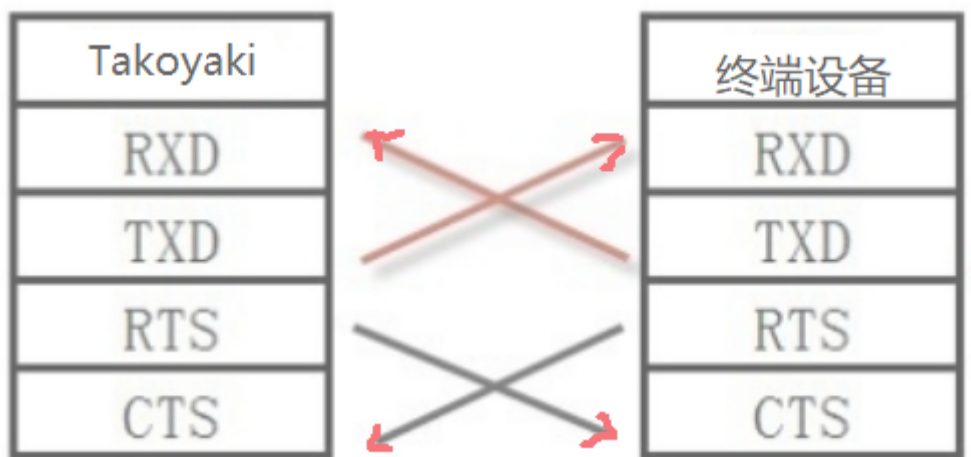


SSD_FUART flow control use reference

1. Fuart flow control process

1.1. HW connection



1.2. Parameter description

- RTS

Require To Send, sending a request, is an output signal, used to indicate that the device is ready to receive data, the low level is active, and the low level

indicates that the device can receive data.

- CTS

Clear To Send, send permission, is an input signal, used to determine whether data can be sent to the other party, low level is valid, low level indicates that the device can send data to the other party.

1.3. Flow control method

If the local end processes the data stream (receives the data of the other party), set RTS to low level; if the CTS pin of the other device is low level, data can be sent.

If the local end does not process the data stream (does not receive data from the other party), set RTS to high level; if the CTS pin of the other device is high level, stop sending data and wait for the change of CTS pin.

1.4. Notes

1. Confirm pin configuration Fuart

kernel\arch\arm\boot\dts\infinity2m-ssc011a-s01a-padmux-display.dtsi :

```
<PAD_FUART_RX PINMUX_FOR_FUART_MODE_1 MDRV_PUSE_FUART_RX>,
<PAD_FUART_TX PINMUX_FOR_FUART_MODE_1 MDRV_PUSE_FUART_TX>,
<PAD_FUART_CTS PINMUX_FOR_FUART_MODE_1 MDRV_PUSE_FUART_CTS>,
<PAD_FUART_RTS PINMUX_FOR_FUART_MODE_1 MDRV_PUSE_FUART_RTS>,
```

kernel\arch\arm\boot\dts\infinity2m-ssc011a-s01a-padmux-display.dtsi :

```
fuart: uart2@1F220400 {
    compatible = "sstar,uart";
    reg = <0x1F220400 0x100>, <0x1F220600 0x100>;
    interrupts = <GIC_SPI INT_IRQ_FUART IRQ_TYPE_LEVEL_HIGH>,
<GIC_SPI INT_IRQ_URDMA IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&CLK_fuart>;
    dma = <0>;
    sctp_enable = <1>;//rts cts enable is 1
    //pad = <PAD_FUART_RX>;//fuart mode2
```

```

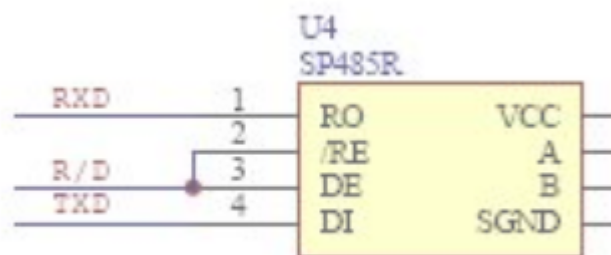
    pad = <PAD_FUART_CTS>;//fuart mode 1
    tolerance = <2>;
    status = "ok";
};

```

2. 确认软件使能cts/rts，如果没有使能cts/rts功能，他们默认值为高电平状态。
3. 如果为了保证ssd20x处于可接受状态，使用命令：`cat /dev/ttyS2 &`，如果没有一直开启ttyS2的设备，那么Fuart的中断状态会自动清除。获取register[101e2]==0x0000;正常为0x0005。

2. 接RS485芯片，半双工通信流程

1. 连接方法



RS485属于master-slaver模式，同时mater作为发起端，salver应答端。网络中同时只有一台在发送数据，其他设备属于监听状态。

主要流程如下, 所有行为需要等待master端的通知。

Host/Master给RS485芯片发送数据时，先把R/D拉高, 发送完毕后，将R/D 拉低，Host/Master进入监听状态，等待slaver应答。

2. app层使用方法

```

struct serial_rs485 rs485conf;

memset(&rs485conf, 0, sizeof(rs485conf));

rs485conf.padding[0] = 17;    //用来控制slaver收发的gpio index

```

```
rs485conf.delay_rts_after_send = 2000; //发送完最后一个字节需要的
delay · 单位：us
rs485conf.flags |= SER_RS485_RTS_ON_SEND; //发送前拉高gpio · 打开
SER_RS485_RTS_AFTER_SEND指的是发送后拉高
rs485conf.flags |= SER_RS485_ENABLED; // 使能本串口485模式 · 默认禁
用
ioctl( iHandle , TIOCSRS485 , & rs485conf );
```

[485pin.c](#) [media/485pin.c]